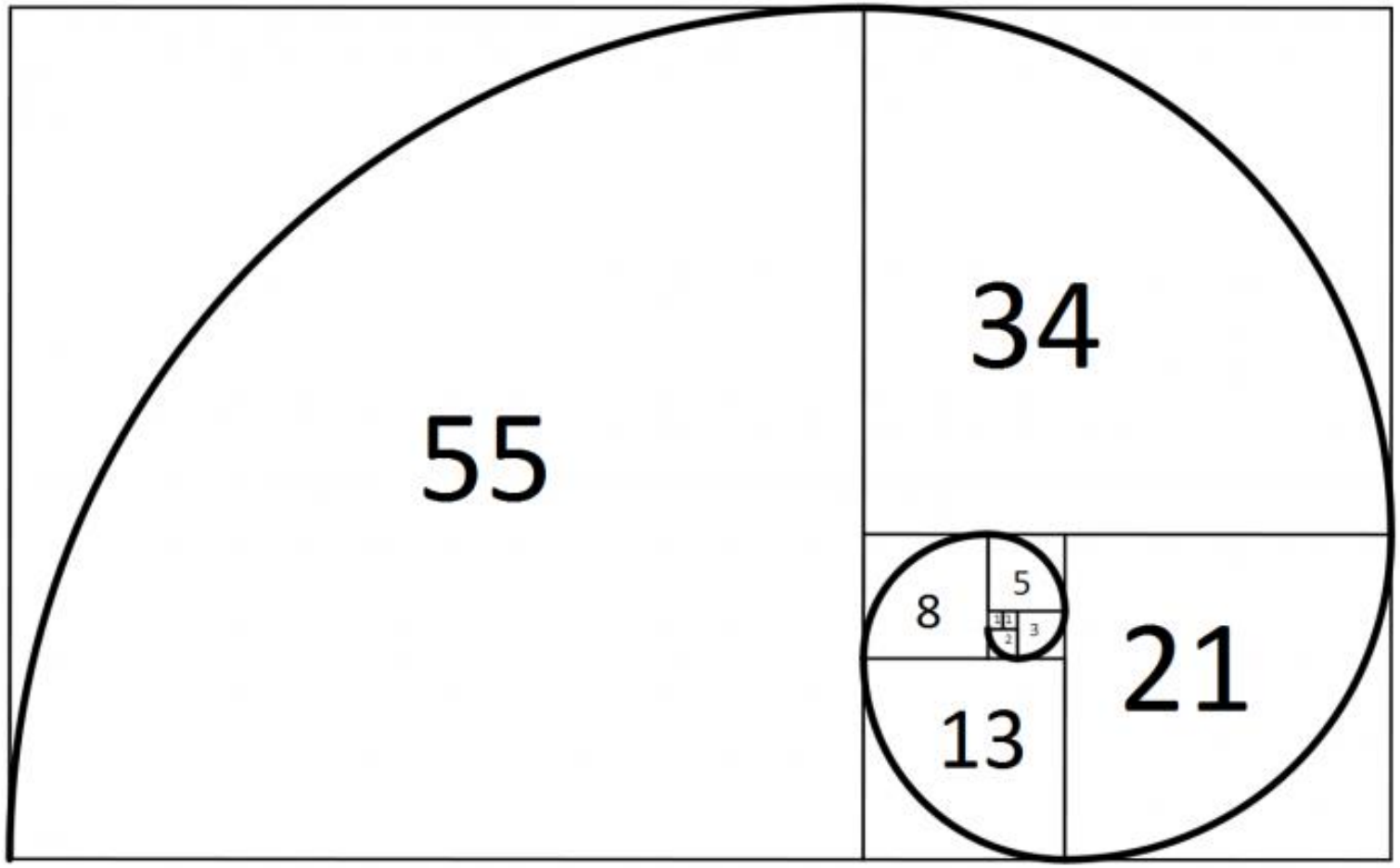
Programming: Recursive functions

Recursive function

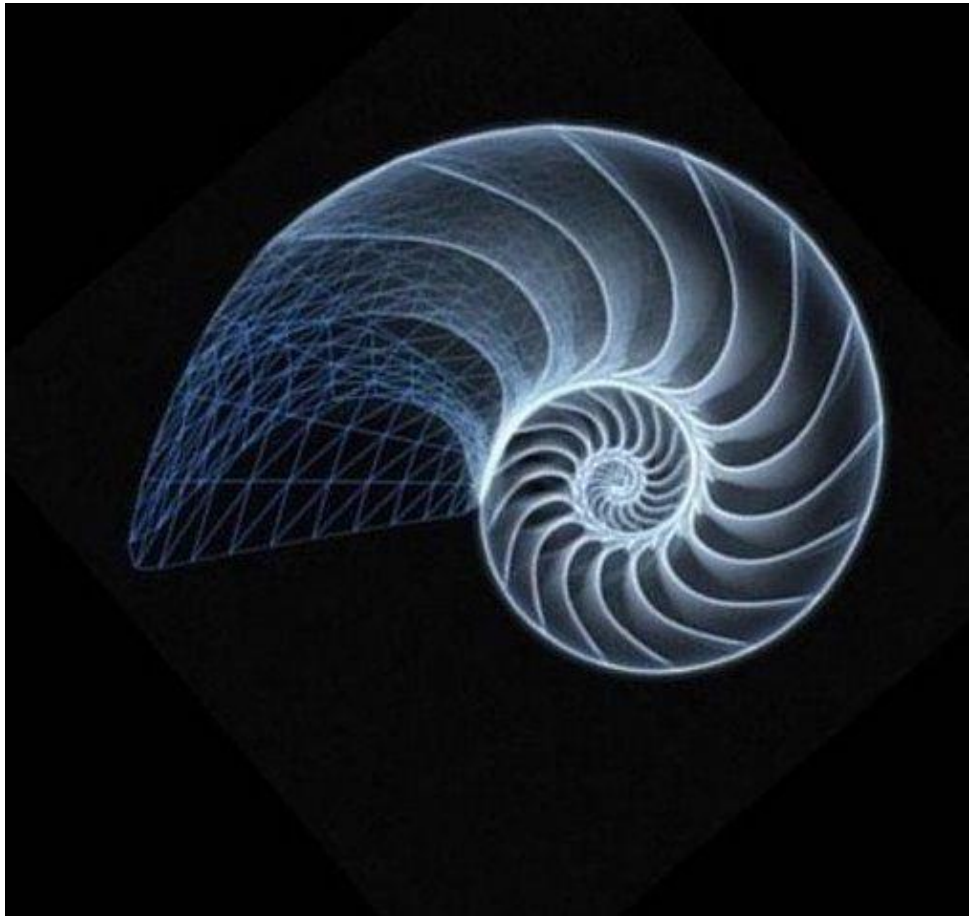
- A function that *calls itself*
- In mathematics there are many examples of functions with a recursive definition
  - n! = n * (n-1)! if n>1
    0! = 1
  - fibonacci(n) = fibonacci(n-1)+fibonacci(n-2) if n>2
    fibonacci(1) = 1 fibonacci(2) = 1
    - *curiosity: in most of the flowers the number of petals belongs to the Fibonacci's sequence*
  - determinant of a square matrix with Laplace formula

$$\det M = \sum_{j=1}^{n} (-1)^{i+j} \det M_{ij}$$

Curiosity: graphical representation of the Fibonacci series



Curiosity: graphical representation of the Fibonacci series



Recursive function (ii)

- The recursive call has always *smaller parameters* (in some sense)
  - The parameter of the recursive call is a smaller number
  - The parameter of the recursive call is a smaller matrix
  - …
- There is always at least one *base case* when the function is computed without recursion
- It is quite easy write a program directly from the recursive definition

Factorial – iterative version

n! = n * (n-1)! if n>1
0! = 1

parameter: n non negative integer
use: i: counter
fact: accumulator
algo:

- catch special case n=0
- set fact to 1 (neutral for *)
- repeat varying i from 2 to n
  - set fact to fact * i
- return fact

```
factorialIte <- function(n){
  if (n<0)
    return(NULL)
  if (n == 0)
    return(1)
  f <- 1
  for (i in 1:n)
     f <- f*i
  return(f)
}
```

Factorial – recursive version

$n! = n * (n-1)!$ if $n>1$
$0! = 1$

```
factorialRec <- function(n){
  if (n<0)
    return(NULL)
  if (n == 0)
    return(1)
  return(n * factorialRec(n-1))
}
```

Fibonacci – recursive version

fibonacci(0) = 0

fibonacci(1) = 1

```
fibonacciRec <- function(n){
if (n < 0)
    return(NULL)
  if (n==0)
    return 0
  if (n == 1)
    return(1)
  return(fibonacciRec(n-1) +
         fibonacciRec(n-2)
        )
}
```

Calls to compute fibonacci(6)

- fib(6)
  - fib(5)
    - fib(4)
      - fib(3)
        - fib(2)
          - fib(1)
          - fib(0)
        - fib(1)
      - fib(2)
        - fib(1)
        - fib(0)
    - fib(3)
      - fib(2)
        - fib(1)
        - fib(0)
      - fib(1)
  - fib(4)
    - fib(3)
      - fib(2)
        - fib(1)
        - fib(0)
      - fib(1)
    - fib(2)
      - fib(1)
      - fib(0)

not efficient, since many computations are repeated

Recursive vs iterative

- Recursive programming is easy to understand
  - once you have understood the "trick"
- In general, a problem that is recursively defined allows an iterative solution
  - sometimes it requires some work to be designed
- It is not straightforward to say if the iterative solution is better or worse than the recursive one….
- Sometimes the recursive one is by far worse
- e.g. fibonacci

Fibonacci – iterative version

- the recursive calls are substituted by a loop
- compute fibonacci(6)

n_2	n_1	n=n_1+n_2				
1	1	2	3	5	8	13

Fibonacci – iterative version

- the recursive calls are substituted by a loop

	n_2	n_1	n=n_1+n_2			
1	1	2	3	5	8	13

Fibonacci – iterative version

- the recursive calls are substituted by a loop

		n_2	n_1	n=n_1+n_2		
1	**1**	**2**	**3**	**5**	**8**	**13**

Fibonacci – iterative version

- the recursive calls are substituted by a loop

			n_2	n_1	n=n_1+n_2	
1	**1**	**2**	**3**	**5**	**8**	**13**

Fibonacci – iterative version

- the recursive calls are substituted by a loop
- compare the number of iterations with the number of calls in the iterative version

				n_2	n_1	n=n_1+n_2
1	**1**	**2**	**3**	**5**	**8**	**13**

Fibonacci – iterative version

- the recursive calls are substituted by a loop
- compare the number of recursive calls and the number of loop executions…

```
Fibonacci - iterative version
Parameter: n, non-negative integer
Uses:
- n_1, n_2: two previous Fibonacci's numbers
Algo:
- if n is non positive returns NULL
- if n is 1 or 2 returns 1
- initialize n_1 and n_2 to 1
- repeat n-2 times
  - compute f as n_1 + n-2
  - set n_2 to n_1
  - set n_1 to f
- return f
```